Deep Learning Project: Chest X-ray Pneumonia Classification

Gorkem Baslik

Università degli Studi di Milano Statale
gorkem.baslik@studenti.unimi.it

March 19, 2026

1 Dataset Version and Considered Data

The dataset used in this project is sourced from Kaggle under the identifier https://www.kaggle.com/datasets/yusufmurtaza01/chest-xray-pneumonia-balanced-dataset. It contains chest X-ray images categorized into two classes: **NORMAL** and **PNEUMONIA**. The dataset was accessed on **March 14, 2026**. While the pipeline supports a "quick mode" subsampling technique to enforce strict runtime limits on constrained environments such as Google Colab, the experiments and results detailed in this report represent execution on the **full dataset** structure to demonstrate final predictive capability.

2 Data Organization

To stream the data dynamically during runtime, the project utilizes the official Kaggle API to download and extract the dataset, eliminating the need to track large file artifacts in the source repository. Once extracted, a custom indexing script recursively crawls the directory tree, mapping each valid image file (`.jpg`, `.jpeg`, `.png`, `.bmp`) to its respective label based on the parent folder name.

The complete corpus is subsequently divided using a **stratified split** into three partitions, strictly maintaining the class balance:

- **Training Set:** 70% of the data.
- **Validation Set:** 15% of the data, used for hyperparameter tuning and early stopping.
- **Test Set:** 15% of the data, isolated for final unbiased evaluation.

To ensure reproducibility, the precise file paths and labels for each split are recorded and exported permanently to `splits/train.csv`, `splits/val.csv`, and `splits/test.csv`.

3 Pre-processing

Prior to entering the neural network, raw images undergo a sequence of transformations mapping them to the expected input domain:

1. **Resizing:** All images are uniformly resized to 224×224 pixels to match the expected input resolution of the backbone network.
2. **Data Augmentation (Training Only):** To combat overfitting and improve model generalization, multiple spatial and color jitter enhancements are randomly applied during training:
   - Random Horizontal Flip ($p = 0.5$)
   - Random Rotation (up to $\pm 10°$)
   - Color Jittering (Brightness: ± 0.1, Contrast: ± 0.1)
3. **Normalization:** After conversion to PyTorch tensors, inputs are standardized using the ImageNet pre-trained normalization parameters (mean: [0.485, 0.456, 0.406], std: [0.229, 0.224, 0.225]).

Sanity visualization checks are included in the implementation pipeline to ensure artifacts apply meaningfully.

4 Algorithms and Implementation

The core predictive algorithm leverages **Transfer Learning**. Specifically, we utilize a **DenseNet-121** architecture, pre-trained on ImageNet. This network effectively mitigates the vanishing-gradient problem and enforces feature propagation through dense block connections, thus establishing a strong baseline model for medical imaging tasks.

Implementation specifics include:

- **Classifier Adaptation:** The original 1000-class fully connected layer is replaced by a simplified `nn.Linear(in_features, 2)` structure outputting logits for the robust binary decision (NORMAL vs PNEUMONIA).
- **Fine-tuning Strategy:** By default, early features are frozen while parameters in deeper structural elements (e.g., `denseblock4` and `norm5`) can be unforced according to the hyperparameter search mechanism.
- **Optimization:** A standard `CrossEntropyLoss` coordinates penalization, actively minimized by the `AdamW` optimizer.
- **Early Stopping:** To prevent overfitting, validation F1 scores are continuously monitored. Optimization is halted if the metric fails to improve over 3 full epochs (patience = 3).

Performance tracking heavily prioritizes the **F1 Score** to holistically evaluate precision and recall balance, appended by tracking for Accuracy and ROC-AUC.

5 Scalability

The programmatic pipeline is strictly designed to fulfill requirements on scalability:

- **Quick Mode Switch:** A globally configurable `quick_mode` parameter enforces strict limits on available samples (e.g., maximum 1200 train images, 250 validation, 250 test per class) per run. This allows rapid prototyping under the strict execution limits bound by Google Colab instances without code restructuring.
- **Full Data Execution:** Switching `quick_mode = False` directs the complete logic back equivalently against the full unconstrained dataset volume.
- **Computing Adjustments:** Configurable constants such as batch size (32 default) and worker counts implicitly scale load across distributed environments or standard multicore CPU deployments efficiently.

6 Experiments

Model selection was conducted under a constrained 4-trial budget navigating different critical hyperparameter setups. The search bounds assessed interactions between Learning Rate (`lr`), Weight Decay, Dropout regularizations, and the state of the final DenseNet feature blocks (Unfrozen vs Frozen).

Table 1 summarizes tuning performance natively recorded by the logging hooks.

Table 1. Hyperparameter Search Trials

Trial	Dropout	LR	Weight Decay	Unfreeze	Val F1	Epochs	Time (s)
1	0.2	1e-4	1e-4	True	**0.9835**	8	1136
2	0.3	3e-4	1e-5	True	0.9820	8	1148
3	0.4	1e-4	1e-5	True	0.9773	5	775
4	0.2	5e-4	1e-4	False	0.9354	8	1147

The optimal selection achieved a best **Validation F1 of** ~ 0.9835, emerging from unfreezing the final feature blocks and restricting to a 1×10^{-4} learning rate mapped alongside dropout 0.2.

7 Results and Discussion

Evaluating the definitive best model (from Trial 1) across the isolated 1280-image Test Split yields striking robustness:

- **Accuracy:** 98.20%
- **F1-score:** 98.20%
- **Precision:** 98.58%
- **Recall:** 97.81%
- **ROC AUC:** 0.9974

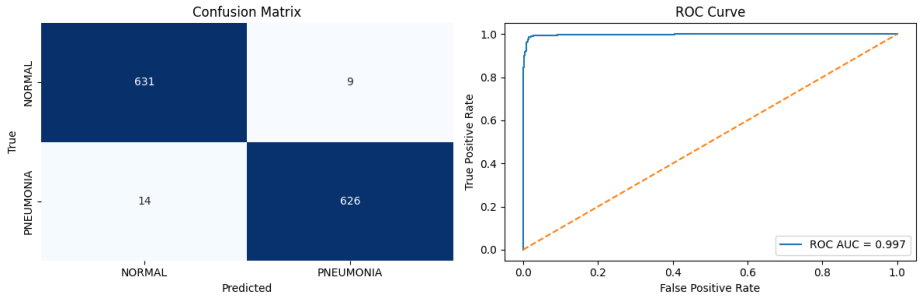

Fig. 1. Confusion Matrix and ROC Curve summarizing test set evaluation performance.

The highly symmetric classification profile validates effective abstraction and minimal class bias. As indicated continuously over the detailed classification report, predicting both Normal and Pneumonia subsets yielded analogous performance characteristics (Normal Recall: 99%, Pneumonia Recall: 98%).

Explainability (Grad-CAM): To trace network behavior, **Grad-CAM visualization** was utilized, intersecting output feature activations overlapping original X-Ray films (see Figure 2). True Positive cases demonstrate correct activation localization toward lungs revealing symptomatic consolidations. False Positives instances occasionally hallucinate non-specific artifacts (e.g., skeletal structures overlapping lung periphery) mimicking ground-glass opacity behavior typical of pneumonia.

8 Reproducibility Notes

To guarantee repeatable results, follow these critical measures:

- Full code is available on GitHub: https://github.com/gorkembaslik/deep-learning
- Set `Config.seed = 42`.
- Due to constraints around public repositories, actual Kaggle API keys reside as `xxxxxx` string placeholders in the root submission block. User credentials must be replaced manually before rerunning Google Colab cells.
- Ensure `quick_mode = False` dictates global behavior to duplicate the full data distribution.

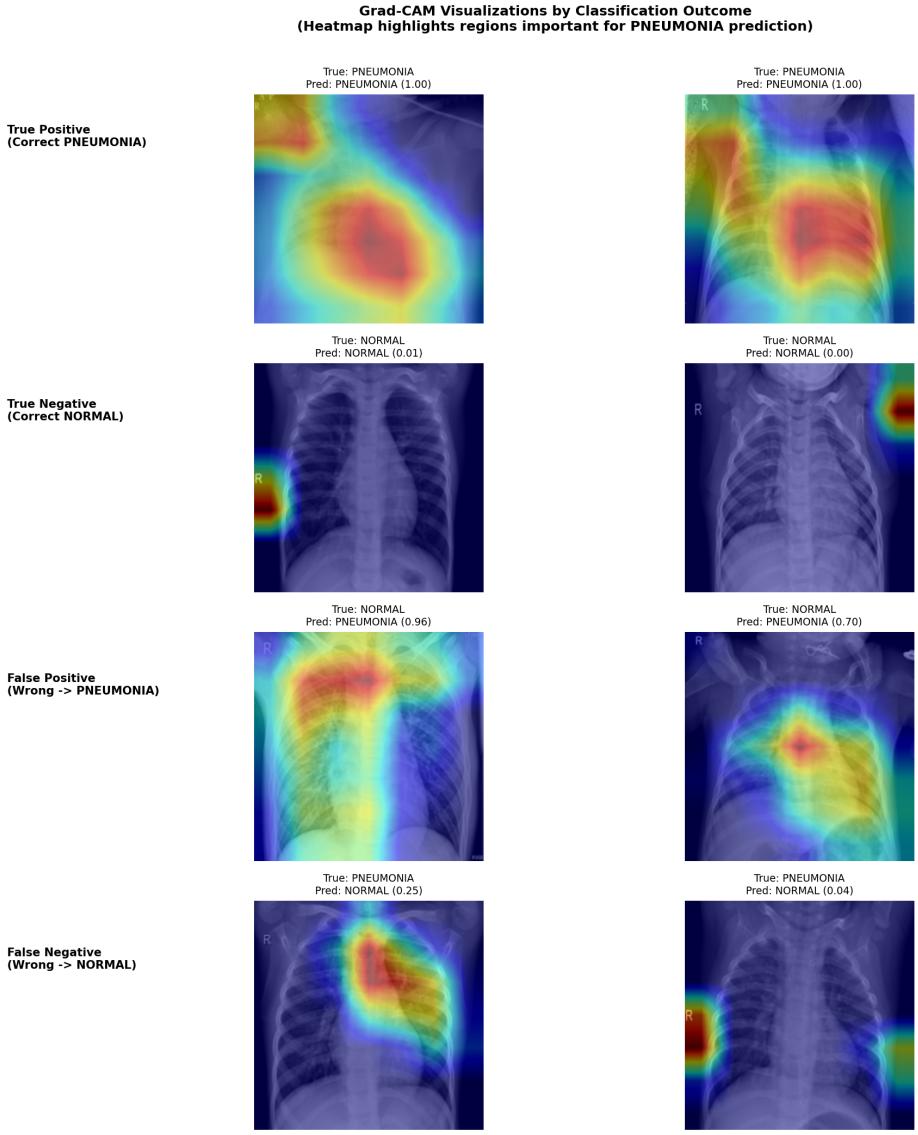

Fig. 2. Grad-CAM Visualizations for True/False Positives and Negatives highlighting important regions for prediction.

I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my/our work. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.